

```
1  #include <iostream>
2  #include <cstdlib>
3  #include <stdio.h>
4  #include "Format.h"
5  #include "Element.hpp"
6  #include "Node.hpp"
7  #include "InitAndTerm.h"
8  #include "DataControl.h"
9  using namespace std;
10 extern DataControl *cDct;
11
12 int InitAndTerm::printMenu()
13 {
14     int num, res;
15
16     while(1){
17         puts("");
18         puts("1.insert");
19         puts("2.delete");
20         puts("3.print");
21         puts("4.close");
22         printf("menu select: ");
23         scanf("%d", &num);
24         while (getchar() != '\n');
25         puts("");
26         if (num > 0 && num < (FUNCTION_COUNT+1)) {
27             cDct->exeFunction((num - 1), res);
28             cout << "execute result: " << res << endl;
29         }else{
30             cout << "please select agin." << endl;
31             continue;
32         }
33         if (num == FUNCTION_COUNT) {
34             return 0;
35         }
36     }
37     return 0;
38 }
39 void InitAndTerm::loadData(Node<Format> *head, Node<Format> *tail) {
40     FILE *fp;
41     int num = -1, age = -1, cnt=0;
42     char tempName[NAME_LENGTH] = { '\0', };
43     Element<Format> *oldData, *newData;
44     const char *savedFile = "file.txt";
45     if ((fp = fopen(savedFile, "r")) == NULL) {
46         cout << "load file not Exist" << endl;
47         return;
48     }
49     char c[1] = { '\0', };
50     Format data(-1, -1, c);
51     newData = new Element<Format>(data);
52     head->setNext(newData);
53     newData->setPrev(static_cast<Element<Format>*>(head));
54
55     while (fscanf(fp, "%d %s %d", &num
56         , tempName, &age) != EOF) {
57         data.setMember(num, age, tempName);
58
59         newData->setData(data);
60         cnt++;
61         head->setCnt(cnt);
62         tail->setCnt(cnt);
63         oldData = newData;
64
65         newData = new Element<Format>(data);
66         oldData->setNext(newData);
67         newData->setPrev(oldData);
68     }
69
70     if (head->getCnt() == 0) {
71         head->setNext(static_cast<Element<Format>*>(tail));
72         tail->setPrev(static_cast<Element<Format>*>(head));
73     } else {
74         oldData->setNext(static_cast<Element<Format>*>(tail));
75         tail->setPrev(oldData);
76     }
77
78     delete newData;
79     fclose(fp);
80 }
81 void InitAndTerm::saveData(Node<Format> *head, Node<Format> *tail) {
82     FILE *fp;
83     Element<Format> *list;
84     const char *fileName = "file.txt";
85     if ((fp = fopen(fileName, "wt")) == NULL) {
86         cout << "file write error" << endl;
87         return;
88     }
89     list = head->getNext();
90     while (list != static_cast<Element<Format>*>(tail)) {
91         fprintf(fp, "%d %s %d\n", list->getData().getNum(),
92             list->getData().getName(), list->getData().getAge() );
93         list = list->getNext();
94     }
95     fclose(fp);
96 }
97 Node<Format>* InitAndTerm::createHeadTailNode()
98 {
99     char dummyName[10] = "dummy";
100     Format data(-1, -1, dummyName);
101     Node<Format>* dummy = new Node<Format>(data);
102     return dummy;
103 }
104 int InitAndTerm::releaseNode(Node<Format> *head, Node<Format> *tail)
105 {
106     Element<Format> *list, *oldData;
107     list = head->getNext();
108
109     while (list != static_cast<Element<Format>*>(tail)) {
110         oldData = list;
111         list = list->getNext();
112         delete oldData;
113     }
114
115     delete head;
116     delete tail;
117     return 0;
118 }
119
```